

Summary of Key Takeaways

1. Computing the Required SRAM Size

To efficiently implement the FlashAttention algorithm on GPUs, it's crucial to calculate the required on-chip shared memory (SRAM) per thread block. This ensures that all necessary data fits within the hardware constraints for optimal performance.

Formula to Compute Required SRAM Size

The total shared memory needed per thread block is calculated by summing the sizes of all variables stored in shared memory during execution:

$$\text{Total Shared Memory (Bytes)} = t \times [2(B_r \times d) + 2(B_c \times d) + (B_r \times B_c) + 2B_r] \quad (1)$$

Where:

- t : Size of each data element in bytes (e.g., $t = 4$ bytes for `float32`, $t = 2$ bytes for `float16`).
- B_r : Block size for queries (number of query vectors processed per block).
- B_c : Block size for keys and values (number of key/value vectors processed per block).
- d : Hidden dimension (feature size).

Variables Stored in Shared Memory

1. **Queries ($\mathbf{Q}_i$)**: $B_r \times d$
2. **Keys ($\mathbf{K}_j$)**: $B_c \times d$
3. **Values ($\mathbf{V}_j$)**: $B_c \times d$
4. **Partial Output ($\mathbf{O}_i$)**: $B_r \times d$
5. **Maxima (m_i)**: B_r
6. **Scaling Factors (ℓ_i)**: B_r
7. **Attention Scores / Normalized Weights ($\mathbf{S}_{ij}$ / $\tilde{\mathbf{P}}_{ij}$)**: $B_r \times B_c$

Steps to Compute Required SRAM

1. **Calculate the total elements needed** using the formula.
2. **Multiply by the element size t** to get the total bytes.
3. **Ensure the total shared memory required does not exceed the GPU's shared memory limit per block** (e.g., 48 KB, 96 KB, or 100 KB).
4. **Adjust B_r and B_c if necessary** to fit within hardware limits.

Example Calculation

For $B_r = B_c = 64$, $d = 128$, $t = 2$ bytes (using `float16`):

1. **Total Elements:**

$$\begin{aligned}\text{Total Elements} &= 2(64 \times 128) + 2(64 \times 128) + (64 \times 64) + 2 \times 64 \\ &= 16,384 + 16,384 + 4,096 + 128 = 36,992\end{aligned}$$

2. **Total Shared Memory (Bytes):**

$$\text{Total Shared Memory} = 2 \times 36,992 = 73,984 \text{ bytes}$$

2. Defining block_dim and grid_dim

Efficient CUDA implementation requires appropriate mapping of computations to CUDA threads and blocks, maximizing parallelism while respecting hardware constraints.

Defining block_dim (Threads per Block)

- **Use Two-Dimensional Blocks:** For matrix operations, a 2D thread block is effective.
- **Typical Block Dimensions:**
 - T_x : Number of threads along the output width (columns).
 - T_y : Number of threads along the output height (rows).
 - Common choices include (16, 16), (32, 8), resulting in 256 threads per block.
- **Constraints:**
 - The total number of threads per block should not exceed 1024.
 - Optimal performance is achieved when the number of threads per block is a multiple of the warp size (32).

Defining grid_dim (Number of Blocks)

- **Compute Grid Dimensions:**

$$\text{grid_dim.x} = \left\lceil \frac{B_c}{T_x} \right\rceil, \quad \text{grid_dim.y} = \left\lceil \frac{B_r}{T_y} \right\rceil$$

- **Each Block's Responsibility:**

- Processes a tile of size $T_y \times T_x$ within the larger $B_r \times B_c$ block.
- Collaborates using shared memory to compute necessary operations efficiently.

Mapping Computations to Threads

- **Threads within a Block:**

- **Row Index:** $\text{row} = \text{blockIdx.y} \times T_y + \text{threadIdx.y}$
- **Column Index:** $\text{col} = \text{blockIdx.x} \times T_x + \text{threadIdx.x}$

- **Per-Thread Workload:**

- Each thread computes or accumulates results for specific elements of the output tile.
- Utilize shared memory to cache data and reduce global memory accesses.

Example Configuration

For $B_r = B_c = 64$, $d = 128$:

- **Block Dimensions:**

```
1 const int T_x = 16;  
2 const int T_y = 16;  
3 dim3 block_dim(T_x, T_y); // 256 threads per block
```

- **Grid Dimensions:**

```
1 const int G_x = (64 + 16 - 1) / 16 = 4;  
2 const int G_y = (64 + 16 - 1) / 16 = 4;  
3 dim3 grid_dim(G_x, G_y);
```

- **Kernel Launch:**

```
1 my_kernel<<<grid_dim, block_dim, shared_memory_size>>>(...);
```

3. Computing Row-Wise Softmax with Multiple Threads on the Same Row

Efficient computation of the row-wise softmax function in parallel requires collaboration among threads to perform reductions and normalization.

Challenges

- **Parallel Reduction for Maxima and Sums:**

- Computing the maximum and sum of elements in a row is necessary for softmax.
- Requires synchronization and efficient reduction algorithms.

- **Avoiding Race Conditions:**

- Threads updating shared variables must be carefully synchronized.

Strategies for Efficient Parallel Softmax

1. Use Parallel Reduction for Row Maxima:

- **Step 1:** Each thread computes the maximum of its assigned elements.
- **Step 2:** Perform a parallel reduction within the warp or block to find the overall maximum for the row.
- **Implementation:** Use shared memory to store intermediate maxima and apply reduction patterns (e.g., tree-based reduction).

2. Compute Exponentials and Partial Sums:

- **Exponentials:** Each thread computes $\exp(\text{element} - \text{row_max})$ for its elements.
- **Partial Sums:** Accumulate the sum of exponentials using parallel reduction.

3. Normalize the Softmax Output:

- **Compute Softmax Values:** Each thread divides its exponentiated elements by the total sum to obtain softmax probabilities.
- **Avoid Divergence:** Ensure all threads within a warp follow the same execution path to prevent performance loss due to divergence.

4. Synchronization:

- Use `__syncthreads()` to synchronize threads after each reduction step.
- Ensure that shared memory writes are visible to all threads before proceeding.

Example Implementation in CUDA

```
1 // Assume each thread processes one element per row
2 __shared__ float row_max[T_y];
3 __shared__ float row_sum[T_y];
4
5 // Step 1: Compute local maxima
6 float local_max = ...; // Compute max over assigned elements
7 // Perform parallel reduction to find row_max[threadIdx.y]
8
9 // Step 2: Compute exponentials and partial sums
10 float exp_val = exp(element - row_max[threadIdx.y]);
11 float local_sum = ...; // Sum of exponentials for assigned elements
12 // Perform parallel reduction to find row_sum[threadIdx.y]
13
14 // Step 3: Normalize to get softmax output
15 float softmax_val = exp_val / row_sum[threadIdx.y];
```

Considerations

- **Avoid Bank Conflicts:** Access shared memory in a way that minimizes conflicts.
- **Use Warp-Level Primitives:** Utilize warp-level intrinsics (e.g., `__shfl_down_sync`) for efficient reductions within warps.
- **Handle Edge Cases:** Ensure correct behavior when the number of elements per row is not a multiple of the warp or block size.

Final Tips

- **Balance Workload and Resources:** Adjust B_r , B_c , and thread block sizes to optimize performance while respecting hardware limits.
- **Optimize Memory Access Patterns:** Ensure coalesced global memory accesses and efficient use of shared memory.
- **Leverage CUDA Features:** Use shared memory, synchronization primitives, and warp-level functions to maximize efficiency.
- **Profile and Iterate:** Use profiling tools to identify bottlenecks and fine-tune your implementation accordingly.

By applying these principles, you can implement the FlashAttention algorithm effectively, ensuring efficient computation of attention mechanisms on GPU architectures.